SYSTEM ARCHITECTURE · BUILDER SPECIFICATION

The Manuscript Architecture

An event-sourced document IR, a deterministic tool layer, and a streaming orchestrator that replace the draft–prompt–draft loop with live, block-level collaboration: confidence scored, provenance attached, and nothing polished twice.

Z.ai — Systems Design

August 29, 2026 · Specification v0.1 · MD / DOCX / PDF / TEX

Table of Contents

1. Problem Statement and Design Goals

The naive workflow - prompt, receive a full draft, read the full draft, prompt again - fails in three specific, repeatable ways. This chapter names those failure modes and derives the design goals that the rest of the specification must satisfy.

1.1 The naive loop and why it emerges

A typical agent-assisted writing session looks deceptively simple: the human states an intent, the model returns a complete document, the human types feedback, and the model returns the complete document again. The loop feels natural because chat interfaces make conversation the only available control surface. It persists because neither side has anywhere else to stand: the model holds no state between turns except the conversation transcript, and the human holds no artifact except the latest draft. Every piece of knowledge about the document - what is settled, what is uncertain, why a sentence reads the way it does - lives inside opaque prose or vanishes between turns.

The consequence is that the document text becomes the only shared database between the human and the agent, and prose is a lossy, unstructured, and expensive medium for that role. The three failure modes below all reduce to this single root cause: state that should live in a system of record is instead smeared across conversation turns and document text. The Manuscript architecture therefore begins by extracting that state into an explicit store, and rebuilds every interaction around it.

1.2 F1 - Redundant convergence (the tenth polishing pass)

Because nothing in the system records the notion of done, the agent re-touches sections the human already accepted. A request like "tighten the introduction" is answered by regenerating not just the introduction but also the conclusion, the headings, and the transitions - each regeneration risks undoing an earlier improvement. Human-approved text has no privileged status in the loop, so the tenth iteration can silently degrade the first draft. The waste is compounded by cost: every polishing pass re-emits the full document even when the intended delta is one paragraph, and by drift: style decisions made in iteration two are forgotten by iteration seven. F1 is a state problem, not a model-quality problem, and it must be solved structurally rather than by asking the model to behave.

1.3 F2 - The opaque draft (documents are bad review vessels)

When the human receives a draft, the only reviewable artifact is finished prose. The reasoning that produced it - which sources were consulted, which claims are solid, which phrasings were considered and rejected - is not represented anywhere the human can see. Reviewing therefore degenerates to either blind trust or a full re-derivation of the argument by the human. Worse, the draft hides its own uncertainty: a confidently-written paragraph and a guessed one are visually identical, so the human cannot allocate scarce attention to the parts most likely to be wrong. What review actually needs is a vessel that exposes the thought process: per-paragraph confidence, the evidence behind each claim, and an affordance to question or freeze any part

independently of the rest.

1.4 F3 - Round-trip starvation (the loop is latency-bound)

The draft-prompt-draft cycle synchronizes two very different clocks. The agent works in seconds-to-minutes bursts; the human reviews in minutes-to-hours. Because feedback can only be delivered after a complete draft exists, and because each new prompt blocks on a full regeneration, the session alternates between idle waiting for one party and bursty overload for the other. The human reads the same material repeatedly just to locate what changed, and the agent pays full-document token cost on every turn. A faster loop needs three properties the naive one lacks: edits should stream in while the human watches, feedback should be deliverable at any moment without halting the agent, and regeneration cost should be proportional to the size of the change rather than the size of the document.

1.5 Design goals

Each failure mode induces a goal, and two additional goals constrain the solution space. The mapping is deliberately one-to-one so that later chapters can be audited against the goals they claim to satisfy. Where a mechanism serves several goals, the primary binding is listed and secondary bindings are noted in the mechanism itself.

Goal	Statement	Fixes	Primary mechanism
G1	Doneness is explicit state: accepted text is immutable by default	F1	Block lifecycle (Ch. 4)
G2	Reasoning is reviewable: confidence, rationale and provenance per block	F2	IR sidecar + confidence UI (Ch. 3, 8, 9)
G3	Interaction is continuous: live streaming edits and non-blocking steering	F3	Orchestrator + SSE live loop (Ch. 5, 9)
G4	All state is deterministic: agents propose, tools dispose	F1, F2	Event-sourced IR store (Ch. 3, 6)
G5	Cognition is pluggable: swap model providers behind one contract	-	ModelAdapter seam (Ch. 7)
G6	Export is lossless: md, docx, pdf, tex are serializations, never sources	F2	Serializer layer (Ch. 10)

Table 1-1 - Design goals and their binding mechanisms

> **Reading guide.** Chapters 3 through 6 specify the deterministic core (store, lifecycle, orchestration, tools). Chapters 7 and 8 specify the intelligence layer that attaches to it. Chapters 9 and 10 specify the human surface and the output pipelines. Chapter 11 maps the specification onto the accompanying working prototype.

2. System Overview and Division of Labor

The system separates concerns along one axis: anything that mutates state is a deterministic tool; anything that generates content is a stateless agent role; anything that coordinates is the orchestrator. LLM calls are confined to the smallest possible surface.

The architecture is organized as five planes. The human interface plane renders the document and collects intent. The orchestrator plane is a single-threaded event loop that owns scheduling and policy. The agent plane contains three stateless roles - planner, drafter, critic - which are pure functions from a context bundle to a structured proposal. The adapter plane isolates model providers behind one interface. The tool plane holds the deterministic components that own every byte of persistent state: the IR store, the retriever, the diff-merge engine, and the exporters. Figure 2-1 shows the module map and the permitted directions of communication.

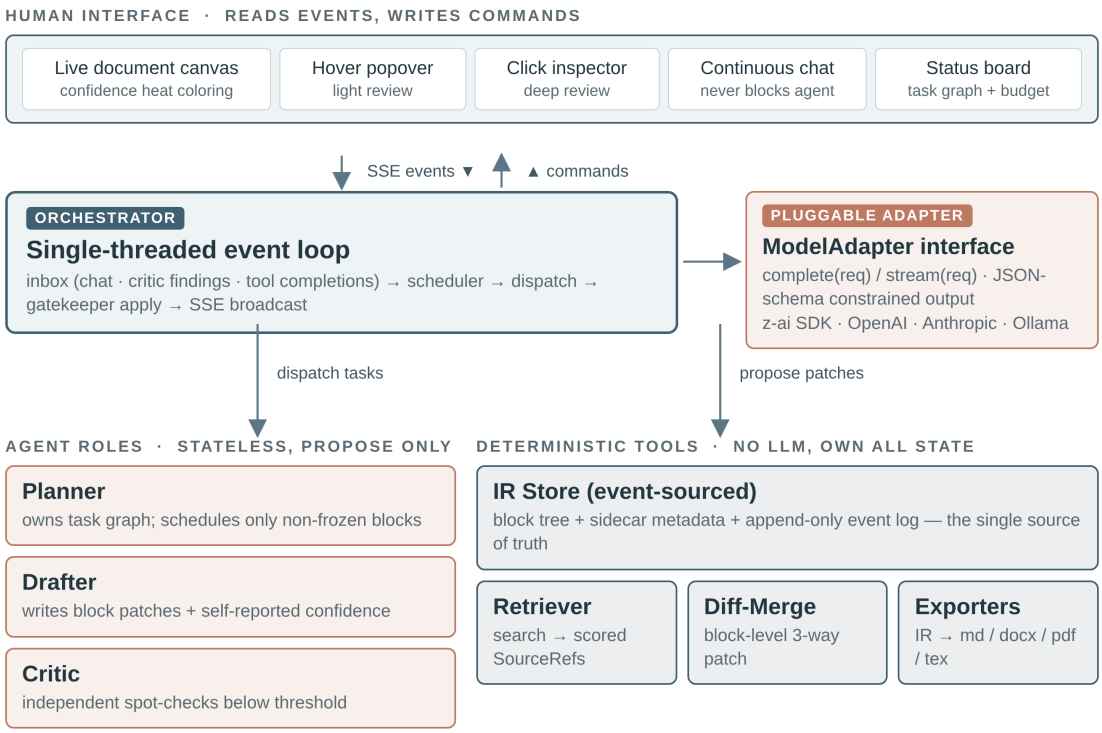

Figure 2-1 - Module map. Solid arrows are direct calls or dispatch; the UI plane receives a broadcast of store events over SSE and issues commands only.

The governing principle is: **agents propose, tools dispose**. An agent can never write to the store, call an exporter, or mutate another agent's output. It returns a typed proposal - a patch, a score, a plan - and the orchestrator decides, through the gatekeeper, whether and how the proposal is applied. This single restriction buys three properties. First, auditability: every mutation is an event with a named origin, so the question "why does the text say X" always has a mechanical answer. Second, safety: a misbehaving model response is rejected at the gate instead of corrupting the document. Third, replayability: because agents are stateless and all effects flow through the event log, a whole session can be replayed from the log alone.

Component	Kind	Owns persistent state	LLM-free
Live canvas / inspector / chat / board	UI	No (projects store events)	Yes
Orchestrator	Process	No (reads event log, emits events)	Yes
Planner / Drafter / Critic	Agent roles	No (stateless proposals)	No
ModelAdapter	Seam	No	No (wraps LLMs)
IR Store	Tool	Yes - blocks, sidecars, events	Yes
Retriever	Tool	Yes - source registry, index	Yes
Diff-Merge	Tool	No (pure function)	Yes
Exporters (md / docx / pdf / tex)	Tool	No (pure serializers)	Yes
Gatekeeper	Tool	No (policy over transitions)	Yes

Table 2-1 - Component inventory and state ownership

2.1 Why the division pays for itself

Keeping every LLM call inside the agent plane means the deterministic core can be tested, reasoned about, and evolved without any model in the loop. The store's transition rules are unit-testable; the exporters are snapshot-testable; the orchestrator's scheduling policy is simulatable against recorded event logs. Conversely, the agent roles are cheap to swap or A/B test because their contract is a schema, not a shared codebase. When a better model appears, only the adapter configuration changes; when a better review policy is devised, only the gatekeeper changes. The division of labor is thus also a division of volatility: volatile intelligence is isolated from stable mechanism.

3. The Manuscript IR: Central Store

One store holds the document as a tree of typed blocks, the review metadata attached to each block, the source registry, and the append-only event log. Markdown, Word, LaTeX and PDF files are serializations of this store - never the other way around.

3.1 Why an AST rather than files

Treating a file (a .docx, a .tex) as the source of truth makes every other subsystem format-aware and brittle: diffing, confidence attachment, live partial updates and review workflows all need parsers for each format, and round-tripping through foreign formats loses structure. The store instead holds a format-neutral intermediate representation: a tree of blocks with stable identifiers. A block is the smallest independently authored, reviewed, and versioned unit - typically a paragraph, a heading, a figure, a table, an equation, a code listing, or a list item. Stable IDs mean that a patch can address "block b_014" across hours of editing, rebasing

and export cycles without re-resolution, and that every piece of sidecar metadata has an anchor that survives rewording.

3.2 Block tree and sidecar metadata

Each block records its type, its content in a canonical inline markup, its parent and ordering index, and a lifecycle status defined in Chapter 4. Attached to every block is a sidecar - the review-oriented metadata that makes the thought process visible. The sidecar is first-class data with the same durability as the content itself; it is not commentary squeezed into the text. The fields below are the minimum viable set discovered by the prototype; the schema is versioned precisely so this list can grow.

Sidecar field	Type	Written by	Purpose
confidence	decimal 0-1 + band	Confidence pipeline (Ch. 8)	Drives heatmap coloring and triage
confidence_parts	s1, s2, s3	Drafter / retriever / critic	Signal breakdown shown in popover
rationale	string	Drafter	One-line "why this text"; hover affordance
alternatives	list of variants	Drafter	Rejected phrasings kept for inspection
open_questions	list of strings	Drafter / critic	Explicit uncertainty surfaced to the human
provenance	block_source links	Retriever / drafter	Which sources back which claim spans
status	enum (Ch. 4)	Gatekeeper only	Lifecycle position; drives scheduling

Table 3-1 - Sidecar metadata attached to every block

3.3 Store schema

The reference schema below is written in SQL for precision; the prototype expresses the same model through Prisma on SQLite. Three design choices matter. The event log is append-only and is the sole write path - current-state tables are projections rebuilt from events. Sidecar values are versioned implicitly through the events that set them, so any earlier review state can be reconstructed. Finally, sources live in their own registry and are linked to blocks through span references, so the same source can back many blocks and coverage can be computed per block (Chapter 8 uses this for signal s2).

Listing 3-1 - Reference store schema (relational notation)

```
-- core identity
documents(id, title, target_format, created_at)
blocks(id, doc_id, parent_id NULL, order_idx,
       type,             -- heading|paragraph|figure|table|equation|code|list
       content,          -- canonical inline markup
       status,           -- placeholder|drafting|draft|in_review|revised|frozen
       updated_at)

-- review sidecar (1:1 with blocks)
block_meta(block_id, confidence, s1, s2, s3, rationale,
           alternatives json, open_questions json, updated_at)

-- provenance
sources(id, doc_id, uri, title, snippet, retrieved_at)
block_sources(block_id, source_id, claim_span)

-- append-only event log (sole write path)
events(id INTEGER PRIMARY KEY, seq, ts, type, actor, payload json)
-- event types: BlockPatched | StatusChanged | CRFiled | CRResolved
--              TaskSpawned | TaskFinished | MessagePosted | BudgetUpdated

-- work and conversation projections
tasks(id, block_id, role, state, cost_tokens, summary)
messages(id, role, block_id NULL, body, ts)
```

3.4 Why event sourcing

Four concrete capabilities fall out of the append-only log, none of which are affordable when the document is a mutable file. **Undo and replay**: any state, including the full sidecar, can be reconstructed by folding events up to a timestamp, so "show me the draft as it was when I asked my question" is a filter, not a convention. **Audit**: every byte of content carries the actor and task that produced it, which is what makes provenance display trustworthy rather than decorative. **Broadcast**: the live UI is a subscriber of the same event stream the orchestrator consumes, so what the human watches is not a rendering of the agent's private state but the actual committed state. **Diverged-history merge**: because events are typed and block-scoped, two lines of work that touched different blocks merge trivially, and the diff-merge tool reduces the pathological cases to explicit conflicts rather than silent overwrites.

4. Block Lifecycle: The Anti-Redundancy Mechanism

Doneness becomes machine-checkable state. The planner can only see blocks that are not yet frozen, and a frozen block re-enters the schedule only through an explicit, human-approved change request. This is the structural kill for F1.

Invariant: the scheduler may only dispatch blocks in non-terminal states. A frozen block is invisible to the planner until a change request is approved — the tenth round of “finishing touches” is structurally impossible.

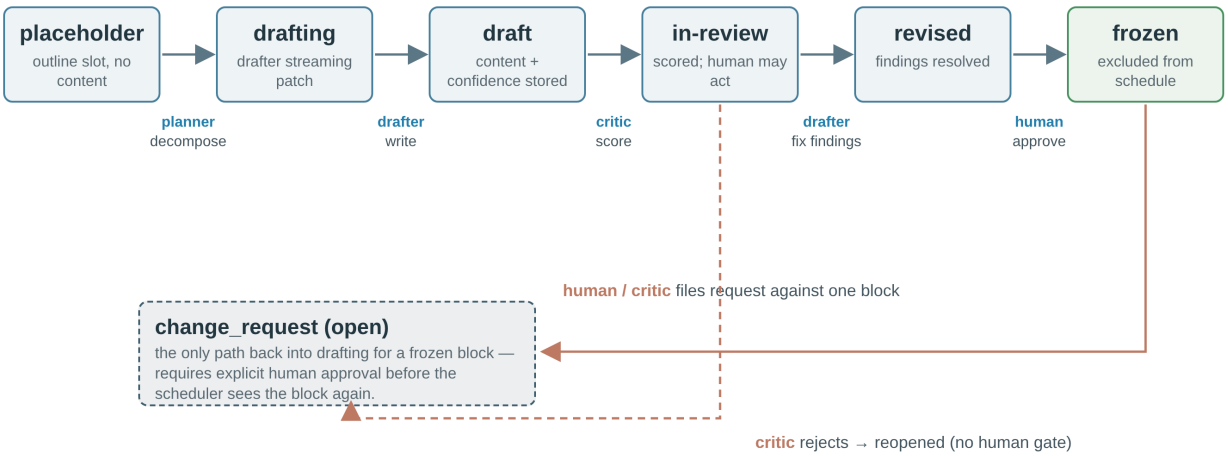

Figure 4-1 - Block lifecycle state machine. Solid arrows are the forward path; orange arrows are the two re-opening paths. Only the gatekeeper may execute a transition.

Every block moves through the same lifecycle. The transitions are executed exclusively by the gatekeeper tool - agents and UI request transitions, and requests that violate the table below are rejected as events. The status is therefore trustworthy to all other components: the scheduler filters on it, the UI decorates on it, and the exporter reports on it. The six states and their semantics are deliberately few; expressiveness was traded for enforceability, and every review nuance that does not fit a state lives in the sidecar instead.

State	Entered via	Exits via	UI treatment
placeholder	planner decomposes outline	drafter begins streaming	dimmed skeleton line
drafting	drafter task starts	drafter finishes patch	animated left edge, streaming text
draft	patch applied	critic scores it	solid text, provisional color
in_review	critic emits score	drafter applies findings, or human acts	confidence band coloring active
revised	findings applied	human approval	highlight on changed spans
frozen	human approve action only	approved change request only	lock glyph in gutter; excluded from schedule

Table 4-1 - Status semantics. Transitions are executed only by the gatekeeper.

4.1 The change-request protocol

A change request (CR) is the single re-entry path for frozen text. A CR is filed by the human (directly, or by rejecting a block in review) or by the critic (a finding against a specific block), and carries four fields: the target block ID, the origin, the requested change in one sentence, and a priority. Filing a CR does not by itself reopen anything - a CR originating from the critic joins a review queue that the human approves or discards, while a CR authored by the human is auto-approved. On approval the block moves back to drafting with the CR attached, and the drafter receives it as part of its context bundle, so the reason for reopening is never separated from the work itself. Resolved CRs remain in the log, which turns the CR queue into a durable record of why the document ever changed after a freeze.

4.2 Why this kills F1 rather than discouraging it

The naive loop allows redundant polish because the cost of re-touching is invisible and the target of a change is the whole document. The lifecycle removes both enablers. Visibility: the planner’s candidate set is defined as blocks whose status is in {placeholder, drafting, draft, in_review, revised} - frozen blocks are not deprioritized but structurally absent, so no amount of model enthusiasm can schedule them. Attribution: re-opening requires a CR with a stated reason, which makes regression visible ("the conclusion changed after you approved it" now names an event). Bounded blast radius: even a legitimate CR re-drafts one block, not the document, so an accepted improvement cannot silently overwrite other accepted text. The tenth polishing pass is not forbidden - it is simply no longer expressible as a single action.

5. Orchestrator: Event Loop and Task Graph

A single-threaded loop consumes the event log, turns human input into block-scoped work, dispatches stateless agent tasks, and applies results through the gatekeeper. It is the only component that talks to everyone.

5.1 The loop

The orchestrator is an event-driven loop with no background threads touching shared state. Each iteration drains the inbox in priority order - interrupts and CR approvals first, then tool completions, then new work - recomputes the task graph from block statuses, and dispatches at most a small bound of concurrent agent tasks. Scheduling boundaries are the points between inbox drain and dispatch; this is where human input that arrived mid-task takes effect without cancelling running work. The loop appends every decision back to the event log, which makes the orchestrator itself auditable and lets a restarted process resume exactly where it stopped.

Listing 5-1 - Orchestrator event loop (pseudocode)

```
loop {
  // 1. drain inbox - priorities: interrupt > CR resolution > tool result > new
  for evt in inbox.drain() { apply(evt) }        // via gatekeeper

  // 2. recompute candidate set from block statuses
  candidates = blocks.where(status != frozen && status != drafting)
  plan = planner.propose(candidates, sidecars, open_crs)   // stateless

  // 3. dispatch within budget and concurrency bounds
  for task in plan.take(MAX_INFLIGHT - inflight) {
    if (!budget.reserve(task.estimate)) break
    tasks.spawn(task); dispatch(task)            // to drafter | critic
  }

  // 4. apply completed proposals (gatekeeper validates each)
  for res in completed() { gatekeeper.apply(res) }

  broadcast(store_events_since(cursor))          // SSE fan-out
}")
```

5.2 Task graph and interruption model

The task graph is derived, not authored: every non-frozen block with unmet needs maps to at most one runnable task (draft, score, or revise), plus explicit tasks for pending change requests. Because derivation is cheap, the graph is recomputed at every boundary instead of being patched incrementally, which eliminates a whole class of scheduler-state bugs. Interruptions come in two grades. **Cooperative** - the default - queues a message or CR for the next boundary; running tasks finish on their own, typically within seconds for a single block. **Pre-emptive** cancels the current agent task before its proposal is applied, reserved for "stop" commands and for human edits landing on a block the drafter is concurrently rewriting. Both grades are

recorded as events, so an interruption is never lost or invisible.

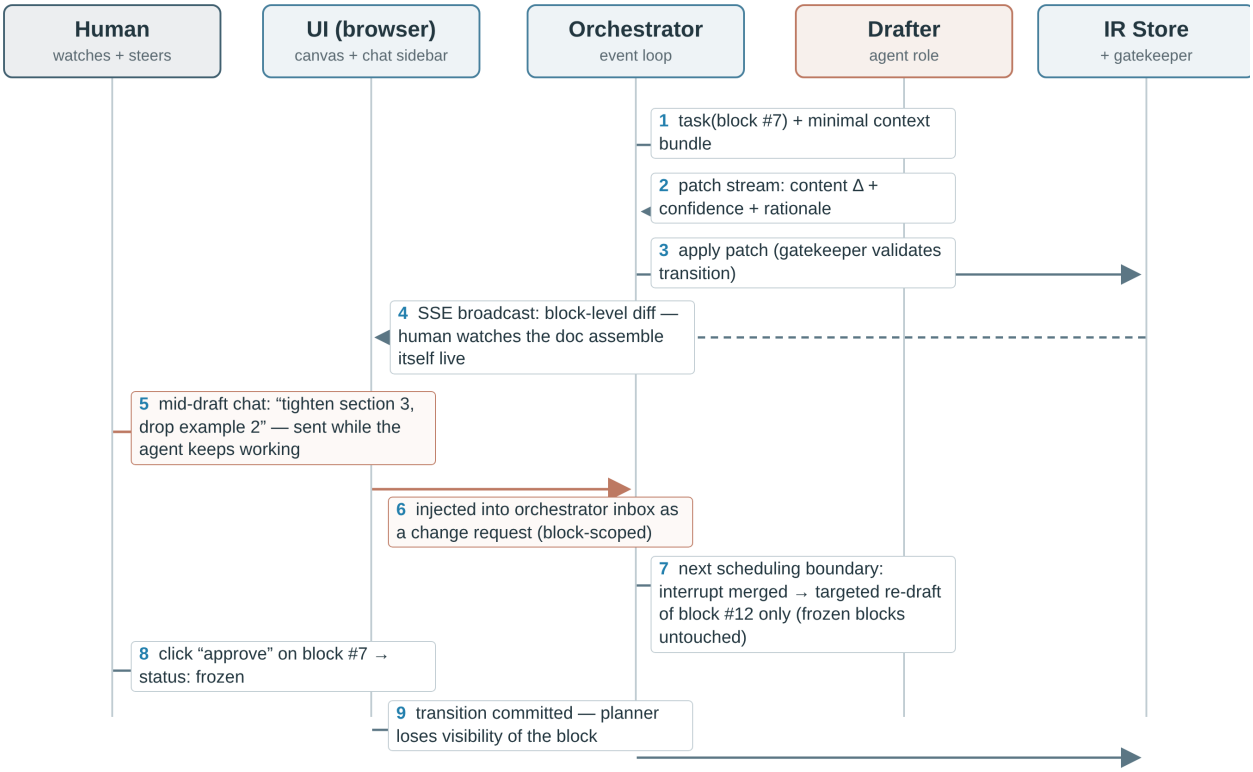

Figure 5-1 - Live loop with a mid-draft interrupt. The human message at step 5 never blocks the drafter; it takes effect at the next scheduling boundary (step 7) as a block-scoped revision.

5.3 Budget guard

The budget guard converts vague reliability hopes into checkable policy. A session carries a total token allowance and per-block retry caps; every dispatch reserves an estimate, every completion reconciles actuals, and BudgetUpdated events make the remaining allowance visible to the UI in real time. When the allowance is exhausted, the orchestrator stops dispatching and reports the best achievable state - which blocks are frozen, which remain in review, and what a continuation would cost. Retry caps interact with the lifecycle: a block that fails critic review twice raises an open question to the human instead of entering a silent regeneration spiral, converting a cost problem into a decision for the person best placed to make it.

6. Tool Layer: Deterministic Contracts

Five tools own all state and all policy enforcement. Each is specified as an interface with invariants and failure semantics, so the agent plane can be evolved without touching them.

The tool layer is deliberately boring software. Every function is total (it either succeeds or returns a typed error), deterministic for a given event log, and free of model calls. The interfaces below are the full surface that agents and the UI are allowed to know about; anything not expressible through them is, by definition, not part of the system.

Listing 6-1 - Tool layer contracts (TypeScript notation)

```
interface IRStore {
  getBlock(id): Block & { meta: Sidecar }
  getNeighborhood(id, radius): Block[]       // for context bundles
  events(since: Seq): StoreEvent[]           // SSE + replay source
  fold(since?: Seq): DocProjection           // current-state view
}

interface Retriever {
  search(query, k): SourceRef[]              // scored, deduplicated
  attach(blockId, refs): void                // claim-span links
  coverage(blockId): number                  // s2 in [0,1]
}

interface DiffMerge {
  patch(base, next): Delta                   // block-scoped diff
  apply(delta): ApplyResult                  // conflict-typed
}

interface Serializer {
  render(doc: DocProjection, fmt: 'md'|'docx'|'pdf'|'tex'): bytes
}

interface Gatekeeper {
  request(actor, transition): Applied | Rejected  // policy check + emit
  validate(proposal: AgentProposal): boolean      // schema + budget + status
}
```

Tool	Key guarantees	On failure
IRStore	Sole write path via events; stable IDs; total ordering	Never partial: an apply is one event
Retriever	Refs always carry URI + span + score; no model call	Coverage degrades to 0 (visible in s2)
DiffMerge	Conflicts typed, never silently resolved	Conflict returns explicit ConflictDelta
Serializer	Export consumes IR only; byte-stable for same fold	Export aborted, no partial file written
Gatekeeper	Only writer of status transitions; policy versioned	Rejected with typed reason, logged as event

Table 6-1 - Tool invariants and failure semantics

Two contracts deserve emphasis. The gatekeeper exists because the most dangerous bug class in agent systems is unauthorized state change - a persuasive model output becoming a document mutation. Routing every transition through one small, tested component makes that bug class unrepresentable rather than merely discouraged. The retriever's coverage function is deliberately exposed as a number: Chapter 8 consumes it as signal s2, and because it is computed from stored span links rather than by asking a model, it is cheap enough to recompute on every relevant event, keeping confidence values live instead of stale.

7. Agent Roles and the Model Adapter

Three stateless roles cover all cognition: planner decomposes, drafter writes, critic verifies. All of them speak through one pluggable adapter contract with schema-constrained responses.

Role	Context bundle (input)	Structured output	Containment
Planner	candidate blocks + sidecars + open CRs + budget state	ordered task list with estimates	proposals only; cannot dispatch
Drafter	target block + neighborhood (radius 2) + pinned chat + CRs + top sources	patch: content delta + s1 + rationale + alternatives + open questions	one block per task; schema-validated
Critic	block under review + its sources + rubric version	s3 + findings list + optional CR suggestion	cannot edit content; may only file CRs

Table 7-1 - Agent role contracts. No role sees the whole document unless it fits in the neighborhood policy.

7.1 Context assembly, not transcript hoarding

Naive sessions degrade because every turn re-sends the whole document and the whole conversation. Here, each task receives an assembled bundle: the target block, its nearest neighbors for cohesion, the chat messages pinned to it, any open change requests, and the small set of retrieved sources the planner judged relevant. The bundle is built by deterministic code from the store, so context is auditable and cost is predictable per block size rather than per document age. Cohesion across distant parts of the document is the planner's job at outline level, not the drafter's burden at paragraph level; a dedicated cohesion pass can still read wider neighborhoods when the planner schedules one explicitly.

Listing 7-1 - ModelAdapter seam. Providers are configuration, not code paths.

```
interface ModelAdapter {
  complete(req: StructuredRequest): Promise;
  stream(req: StructuredRequest): AsyncIterable;
}

interface StructuredRequest {
  role: 'planner' | 'drafter' | 'critic';
  schema: JSONSchema;        // response validated before acceptance
  bundle: ContextBundle;     // assembled by the orchestrator
  toolResults?: ToolResult[]; // retriever outputs made available
}

interface StructuredResponse {
  ok: boolean;
  data: unknown;             // instance of req.schema
  usage: { inputTokens: number; outputTokens: number };
}

// providers implement the same contract:
// ZaiSDKAdapter | OpenAIAdapter | AnthropicAdapter | OllamaAdapter
// MockAdapter: deterministic replay for tests and the demo mode
```

7.2 Token economics of block-scoped work

The economics follow directly from the task shape. A full-document regeneration pays for every block on every turn; a block-scoped patch pays for one block plus a neighborhood of two on each side and its pinned context. For a representative 40-block document the asymmetry is roughly an order of magnitude per revision, and it compounds over a session because approved blocks stop appearing in any context bundle at all. The saving is not an optimization trick but a consequence of G1: frozen text is absent from the working set, so the model is never paid to re-derive decisions that were already made.

≈ 92%

estimated output-token saving of a one-block revision versus full-document regeneration in a 40-block document (0.9k vs 12k output tokens, same neighborhood policy)

8. Confidence Pipeline: Hybrid Signals

Every block carries a composite confidence score assembled from three deliberately different signals: a self-report at write time, a deterministic citation-coverage measure, and a critic spot-check reserved for weak blocks.

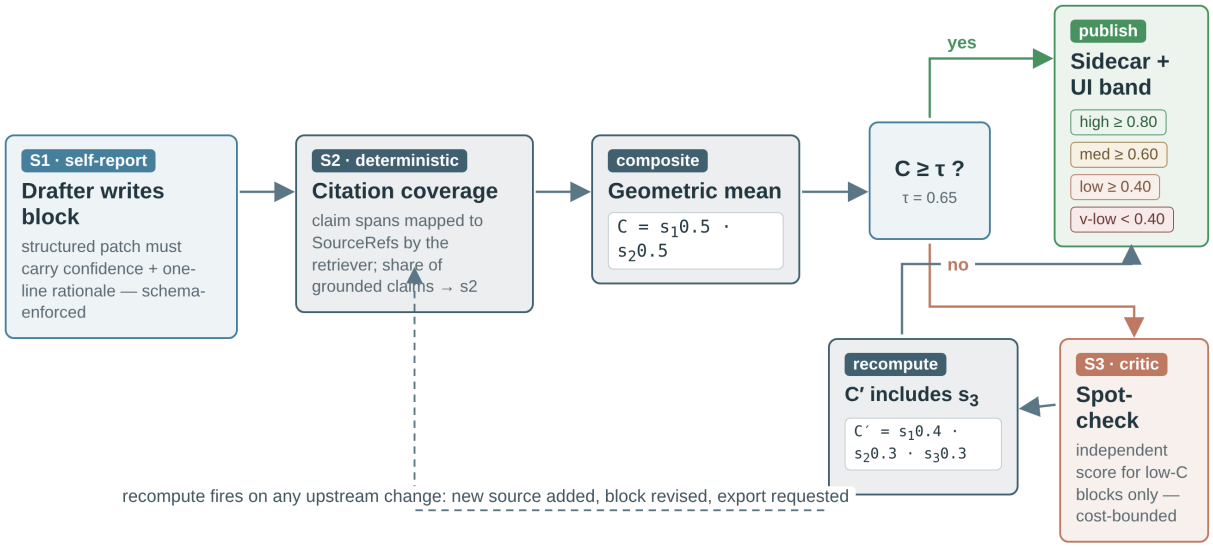

Figure 8-1 - Confidence pipeline. S1 and S2 are computed for every block; the expensive S3 runs only below the threshold, keeping critic cost proportional to doubt.

Signal	Source	Cost	Known bias	Role in composite
s1 self-report	drafter, at patch time (schema field)	zero (bundled)	overconfident under time pressure	ownership: the writer commits to a number
s2 citation coverage	retriever span links (deterministic)	milliseconds	high for well-sourced trivia, low for synthesis	anchor: grounds the score in evidence
s3 critic score	independent verifier pass	one LLM call	correlated with rubric, not with truth	adjudicator: only where s1+s2 disagree with reality

Table 8-1 - The three signals and why all three are needed

The composite is a weighted geometric mean. Geometry beats arithmetic here for one practical reason: a block that is confidently written (s1 = 0.9) but barely sourced (s2 = 0.3) should score as suspect, and a geometric mean punishes imbalance far more than an average does. Weights start equal and are config, not dogma; the calibration loop in 8.2 is the mechanism that adjusts them from evidence. The result is banded into four ranges that map directly onto the UI treatment in Chapter 9, so "what does 0.62 mean" has one answer everywhere in the system.

> **Worked example.** A block drafts with s1 = 0.9 and coverage s2 = 0.6: $C = 0.9^{0.5} \cdot 0.6^{0.5} \approx 0.735$ - medium band, no critic call. A rival block reports s1 = 0.9 but only s2 = 0.3: $C \approx 0.9^{0.5} \cdot 0.3^{0.5} \approx 0.520$ - low band, triggers a critic pass; if the critic returns s3 = 0.5, the recomputed score is $C' = 0.9^{0.4} \cdot 0.3^{0.3} \cdot 0.5^{0.3} \approx 0.547$ - still low, and the block shows up in the triage queue with its open questions attached.

8.1 Recomputation triggers

Confidence is a projection of current evidence, not a verdict rendered once. The pipeline recomputes a block's score when any input changes: a new source is attached or detached, the block is revised, a critic finding lands, or the human edits the text inline (which pins s1 to 1.0 for the human-authored spans - the one place where self-report is ground truth). Recomputation is cheap for s1/s2 and budgeted for s3, and every recompute is an event, so the UI can animate a band change rather than silently repainting. A block whose band drops after a freeze does not silently reopen; it files a CR suggestion for the human, consistent with the Chapter 4 rule that only approved CRs reopen frozen text.

8.2 Calibration loop

Self-reports drift toward optimism unless disciplined. The pipeline keeps the history of (s1, s2, s3) triples per block and, in aggregate, per drafter model and prompt version. When residuals accumulate - say s1 systematically exceeds s3 by 0.15 - the calibration job tightens the drafter's rubric wording or adjusts the s1 weight down for that configuration, and the change is itself an event so the effect is measurable. The goal is not a philosophically perfect probability but a stable, monotone signal: when the page turns redder, there really is more doubt.

9. Live Human Interface

The UI is a projection of the event stream, not a window into the agent. Four surfaces - canvas, popover, inspector, chat - give the human triage, light review, deep review, and steering without ever blocking the system.

9.1 Streaming protocol

The server pushes a single ordered event stream over SSE; every surface is a pure reducer over it. Patch granularity is the block, so a revision re-renders one paragraph while the rest of the page stays visually stable - the property that makes "watching the document write itself" legible rather than frenetic. The protocol carries five event kinds, which is deliberately the same set as the store's event log; the UI consumes the log directly through a thin replay cursor rather than a bespoke API, so nothing shown can diverge from what was committed.

Event	Payload (abridged)	UI reaction
BlockPatched	blockId, content delta, s1, rationale	stream text into the block; flash left edge
StatusChanged	blockId, from, to	gutter glyph + coloring mode switch
ConfidenceUpdated	blockId, composite, parts	band color transition (animated)
MessagePosted	role, body, pinned blockId	chat sidebar append; pin marker in canvas
BudgetUpdated	used, remaining, per-task	status board meter

Table 9-1 - Streaming events and their surface effects

9.2 The four surfaces

The **canvas** renders blocks with two visual channels. Confidence is shown as a left-border tint plus a faint background wash in four low-saturation bands - high (green, ≥ 0.80), medium (khaki, ≥ 0.60), low (orange, ≥ 0.40), very low (red, below 0.40) - chosen so a page reads like a heat map at arm's length without turning into traffic lights. Status is shown in a narrow gutter: a subtle animation while drafting, a solid bar for draft states, and a lock glyph for frozen blocks. Together the two channels answer the reviewer's first two questions - "where should I look" and "what may still change" - before a single click.

The **hover popover** is light review: it shows the confidence number and its three signal parts as small bars, the drafter's one-line rationale, and source count. It appears on hover with zero commitment, making it cheap to sweep a page and sample the agent's state of mind. The **inspector**, opened by click, is deep review: a side panel with tabs for Reasoning (rationale, alternatives considered and why they were rejected), Sources (linked spans, with the retrieved snippet), Questions (open questions with answer boxes that post back as pinned messages), and History (the block's event trail with diffs). The inspector is also where actions live: approve and freeze, reject as a CR, or edit inline.

The **chat sidebar** implements continuous prompting. Messages are sent while the agent works; each is either free-floating or pinned to a block ("about this paragraph"). Pinned messages join that block's next context bundle, which is how "tighten section 3" becomes work on the right blocks rather than a full-document regeneration. A message never pauses the loop: it lands in the inbox and takes effect at the next scheduling boundary (Chapter 5), with pre-emption reserved for explicit stop. The **status board** shows the derived task graph - pending, running, blocked-on-human - together with the budget meter and the CR queue, so "what is it doing and what does it cost" has a standing answer.

> **Failure-mode accounting.** F2 dies because reasoning is now inspectable per block (confidence parts, rationale, alternatives, sources) instead of buried in prose. F3 dies because the human reads patches as they stream and steers mid-flight, instead of re-reading whole drafts, and because block-scoped context keeps each turn's cost proportional to the change.

10. Serialization Pipelines

The IR is the document; files are renders. Each target format has one deterministic serializer, and imports are treated as lossy ingestion, never as authoritative state.

Markdown maps one-to-one from blocks and is the lossless lingua franca: every block type has a canonical markdown rendering, and sidecar metadata can be emitted as an optional JSON front-matter block for round-trips. LaTeX is produced from per-type templates - headings to sections, equations to their environments, figures to includegraphics wrappers - and compiled with a pinned engine for the PDF target, which is preferable to a direct PDF writer because it inherits proper math and bibliography handling. DOCX is generated from the canonical markdown through a pandoc-based pipeline with a reference stylesheet, keeping the docx path a serialization concern rather than a second authoring surface. Citations are resolved at export time from the source registry, so the same document exports consistently regardless of which sources were attached when blocks were drafted.

Block type	md	tex	pdf	docx	Notes
heading	H	H	H	H	level preserved
paragraph	H	H	H	H	inline markup translated per target
figure	H	H	H	H	asset copied to export bundle
table	H	H	H	H	column widths best-effort in docx
equation	P	H	H	P	native tex; image fallback elsewhere
code	H	H	H	H	listing styling from template
list	H	H	H	H	ordered and unordered preserved

Table 10-1 - Block type by format capability. H = faithful, P = partial (visual fidelity preserved, semantics flattened).

10.1 Import policy

Importing a foreign file parses it into blocks and marks every resulting block with confidence null and status in_review - imported text is visibly unassessed rather than spuriously confident. Structure that does not survive the parse (complex fields, exotic environments) is quarantined into code blocks with a loss notice in the open_questions field, so nothing disappears silently. The parser versions are recorded on the document, and re-import is idempotent: the same file folds to the same block IDs, which makes "re-ingest after upstream edits" a diff instead of a duplicate.

11. Prototype Implementation Notes

The accompanying prototype implements the confidence-first slice of this specification end to end: event-sourced IR store, gatekeeper transitions, SSE live loop driven by a deterministic MockAdapter, and the four UI surfaces with heat coloring.

The MVP scope agreed for this build is the confidence UI (G2) together with enough of the live loop (G3) to demonstrate it honestly: a scripted authoring session streams real patches through the real gatekeeper into the real store, and the UI is the production reducer - nothing is faked in the client. Cognition is behind the ModelAdapter seam (G5); the MockAdapter replays a deterministic session so the demo runs with zero credentials, and the ZaiAdapter / OpenAIAdapter slots accept real providers without UI changes. Exporters (G6) are stubbed at the seam: the serializer interface exists and markdown rendering works, while docx/pdf/tex pipelines are Phase 1 work.

Subsystem	Status	Notes
IR store (blocks, sidecar, events)	implemented	Prisma + SQLite; append-only event log
Gatekeeper + lifecycle (Ch. 4)	implemented	all transitions validated and logged
Orchestrator loop + SSE (Ch. 5)	implemented	single node process; cooperative interrupts
MockAdapter session replay	implemented	deterministic; drives the demo
Provider adapters (z-ai / OpenAI / Anthropic)	seam only	interface + config slots, Phase 1 wiring
Confidence UI: canvas + popover + inspector	implemented	4 bands, rationale, sources, alternatives
Continuous chat + CR queue + status board	implemented	pinned messages create block-scoped CRs
Exporters md / docx / pdf / tex	md only	serializer interface fixed; Phase 1

Table 11-1 - Prototype coverage against this specification

11.1 Stack and layout

Concern	Choice
Framework	Next.js 16 (App Router) + TypeScript, single deployable
State of record	Prisma ORM over SQLite (events, blocks, sidecars, CRs, messages)
Live transport	SSE route streaming store events; POST endpoints for commands
UI system	Tailwind CSS + shadcn/ui; Zustand for client event reduction
Adapter seam	src/lib/adapters - MockAdapter today, provider adapters Phase 1

Table 11-2 - Prototype stack

Three demo paths exercise the specification: start a simulated authoring run and watch blocks stream in with their bands; click any block to inspect rationale, sources and alternatives, then reject it and watch the block-scoped CR flow; and approve blocks to freeze them, then send a chat instruction that would have triggered the old tenth-pass polish and observe that frozen text does not move. The prototype's event log exports as JSON, which doubles as a fixture format for testing future orchestrator policies against real session shapes.

12. Risks, Trade-offs and Roadmap

The architecture buys enforceable review semantics at the price of new failure surfaces. Each trade-off below names the mitigation that keeps it honest, and the roadmap sequences adoption so each phase stands alone.

Trade-off / risk	Consequence if ignored	Mitigation
Block granularity vs narrative coherence	locally fine text, globally disjointed argument	planner cohesion passes read wider neighborhoods; outline-level tasks can span blocks
Self-reported confidence miscalibration	heat map drifts optimistic, triage rots	critic residuals per model/prompt feed the calibration loop (8.2); s2 anchors scores
SSE fan-out at scale	many tabs or long docs exhaust the node	single ordered stream per session; later, pub/sub fan-out (Phase 2) behind same protocol
Event log growth	slow folds, big backups	periodic snapshot events; fold(since=snapshot); log archived cold (Phase 2)
Event schema evolution	old sessions unreplayable	versioned event envelopes; projections tolerate unknown types by design
Prompt injection via retrieved sources	store writes proposed by hostile text	schema-validated proposals only; agents have no tool side effects; source sandboxing (Phase 1)

Table 12-1 - Trade-off register with mitigations

12.1 Roadmap

- **P0 - confidence core (complete).** This specification plus the prototype: store, lifecycle, gatekeeper, SSE live loop, MockAdapter session, four UI surfaces.
- **P1 - real cognition and export.** Wire z-ai/OpenAI/Anthropic adapters behind the existing seam; ship docx/pdf/tex serializers; source sandboxing; multi-user auth on the session level.
- **P2 - memory and scale.** Source libraries across documents; review memory (per-block decision history informing future drafts); pub/sub fan-out; event-log snapshots and cold archival.
- **P3 - concurrent authoring.** Multiple drafter instances on disjoint block sets with CR-based merge; cohort critic with rubric versioning; cost dashboards over the event log.

The sequencing principle is that each phase must leave the review semantics intact: P1 adds capability without touching the lifecycle, P2 adds memory without changing the protocol, and P3 parallelizes work that the gatekeeper already mediates. Because every phase consumes and produces the same event log, adoption is additive rather than migrational - the riskiest word in this document remains "draft", and the roadmap is designed so it never needs to mean "the whole document, again".